10-design-patterns

10. Design Patterns & Prinsip SOLID

Level: Beginner–Intermediate
Jam: 8 (4 minggu × 2 sesi)
Prasyarat: Modul OOP (modul 8) atau paham class & interface di TypeScript
Output: Aplikasi Express + Mastra yang di-refactor pake SOLID + design patterns

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Menerapkan 5 prinsip SOLID di TypeScript
- Menggunakan Creational patterns (Singleton, Factory, Builder) di proyek nyata
- Menggunakan Structural patterns (Adapter, Decorator, Proxy)
- Menggunakan Behavioral patterns (Observer, Strategy, Command, Template Method)
- Nulis functional code murni — pure function, immutability, composition
- Menerapkan patterns ke Express routes & Mastra tools

Materi

Sesi	Topik
1	SOLID — SRP, OCP, LSP, ISP, DIP
2	Creational Patterns — Singleton, Factory, Builder
3	Structural & Behavioral Patterns — Adapter, Decorator, Proxy, Observer, Strategy
4	Functional Programming & Real-World Patterns — Pure function, immutability, compositio

Output Akhir Modul

Express API + Mastra agent yang di-refactor pake SOLID & design patterns. Kode bersih, modular, gampang di-test.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- ”Explain this SOLID violation and suggest a fix”
- ”Refactor this class to use Strategy pattern”
- ”Generate before/after code for this Factory pattern”
- ”Show me the pros and cons of using Singleton here”
- ”Create a test for this pure function”
- ”What pattern fits this problem? Why?”

Sesi 1 — Prinsip SOLID

SOLID adalah 5 prinsip desain OOP yang bikin kode gampang dirawat, gampang dikembangin, dan nggak gampang patah pas ada perubahan. Dikenalin Robert C. Martin (Uncle Bob).

S — Single Responsibility Principle (SRP)

Satu class = satu tanggung jawab.
Kalau ada alasan lebih dari satu buat ngubah suatu class, berarti class itu kebanyakan kerjaan.

□ Sebelum (langgar SRP)

```
// Satu class ngurusin penyimpanan + notifikasi
class PesananService {
  simpanPesanan(pesanan: any): void {
    // Simpan ke database
    console.log(`Menyimpan pesanan ${pesanan.id} ke DB...`);
    // Kirim email
    console.log(`Mengirim email konfirmasi ke ${pesanan.email}...`);
    // Kirim WhatsApp
    console.log(`Mengirim WhatsApp ke ${pesanan.phone}...`);
  }
}
```

✅ Sesudah (SRP)

```
class PesananRepository {
  simpan(pesanan: any): void {
    console.log(`Menyimpan pesanan ${pesanan.id} ke DB...`);
  }
}

class EmailService {
  kirimKonfirmasi(email: string): void {
    console.log(`Mengirim email konfirmasi ke ${email}...`);
  }
}

class WhatsAppService {
  kirimNotifikasi(phone: string): void {
    console.log(`Mengirim WhatsApp ke ${phone}...`);
  }
}

// Orchestrator — tapi tetap SRP karena cuma ngatur urutan
class PesananOrchestrator {
  constructor(
    private repo: PesananRepository,
    private email: EmailService,
    private wa: WhatsAppService
  ) {}

  proses(pesanan: any): void {
    this.repo.simpan(pesanan);
    this.email.kirimKonfirmasi(pesanan.email);
    this.wa.kirimNotifikasi(pesanan.phone);
  }
}
```

Pakai SRP kalau: Ada class yang mulai ngurusin banyak hal nggak nyambung — penyimpanan, notifikasi, validasi, logging di satu tempat.

O — Open/Closed Principle (OCP)

> **Tertutup untuk modifikasi, terbuka untuk ekstensi.**
> Jangan utak-atik kode yang udah jalan. Tambahin fitur baru dengan nambah class baru.

❌ Sebelum (langgar OCP)

```ts
function hitungGaji(jenis: string, pokok: number): number {
  if (jenis === 'tetap') return pokok + 500_000 + pokok * 0.1;
  if (jenis === 'kontrak') return pokok + 200_000;
  if (jenis === 'magang') return pokok;
  // setiap tambah jenis, edit fungsi ini — risiko error!
  return pokok;
}
```

✅ Sesudah (OCP)

```ts
interface KalkulatorGaji {
  hitung(pokok: number): number;
}

class GajiTetap implements KalkulatorGaji {
  hitung(pokok: number): number {
    return pokok + 500_000 + pokok * 0.1;
  }
}

class GajiKontrak implements KalkulatorGaji {
  hitung(pokok: number): number {
    return pokok + 200_000;
  }
}

class GajiMagang implements KalkulatorGaji {
  hitung(pokok: number): number {
    return pokok;
  }
}

class Payroll {
  constructor(private kalkulator: KalkulatorGaji) {}

  proses(pokok: number): number {
    return this.kalkulator.hitung(pokok);
  }
}

// Tambah jenis baru tanpa edit code lama:
class GajiFreelance implements KalkulatorGaji {
  hitung(pokok: number): number {
    return pokok + pokok * 0.05;
```

```
  }
}
```

Pakai OCP kalau: Sering nambah varian/jenis baru dan capek edit if-else terus.

L — Liskov Substitution Principle (LSP)

> **Objek anak class harus bisa menggantikan objek parent class tanpa bikin error.**

❌ Sebelum (langgar LSP)

```
class Rekening {
  constructor(protected saldo: number) {}

  tarik(jumlah: number): void {
    if (jumlah <= this.saldo) {
      this.saldo -= jumlah;
      console.log(`Tarik Rp${jumlah}. Sisa: Rp${this.saldo}`);
    }
  }
}

class RekeningTabungan extends Rekening {
  tarik(jumlah: number): void {
    const biayaAdmin = 5000;
    const total = jumlah + biayaAdmin;
    if (total <= this.saldo) {
      this.saldo -= total;
      console.log(`Tarik Rp${jumlah} + admin Rp${biayaAdmin}. Sisa: Rp${this.sal
    }
  }
}

function prosesTarik(rekening: Rekening, jumlah: number): void {
  const saldoAwal = rekening['saldo'];
  rekening.tarik(jumlah);
  // ❌ Kalau RekeningTabungan, saldo akhir beda — breaking substitution
}
```

✅ Sesudah (LSP)

```ts
interface Rekening {
  saldo: number;
  tarik(jumlah: number): void;
}

class RekeningReguler implements Rekening {
  constructor(public saldo: number) {}

  tarik(jumlah: number): void {
    if (jumlah <= this.saldo) {
      this.saldo -= jumlah;
      console.log(`Tarik Rp${jumlah}. Sisa: Rp${this.saldo}`);
    }
  }
}

class RekeningPremium implements Rekening {
  constructor(public saldo: number) {}

  tarik(jumlah: number): void {
    const biayaAdmin = 5000;
    const total = jumlah + biayaAdmin;
    if (total <= this.saldo) {
      this.saldo -= total;
      console.log(`Tarik Rp${jumlah} (admin Rp${biayaAdmin}). Sisa: Rp${this.sal
    }
  }
}

// Fungsi ini kerja buat interface apa pun — LSP aman
function prosesTarik(rekening: Rekening, jumlah: number): void {
  rekening.tarik(jumlah);
}
```

Pakai LSP kalau: Kamu bikin inheritance dan ragu apakah anak class benar-benar ”adalah” parent-nya.

I — Interface Segregation Principle (ISP)

> **Jangan paksa class implement method yang nggak dia butuhin.**

❌ **Sebelum (langgar ISP)**

```ts
interface Worker {
  kerja(): void;
  istirahat(): void;
  makan(): void;
}

// Robot nggak makan, tapi dipaksa implement
class RobotWorker implements Worker {
  kerja(): void {
    console.log('Robot bekerja...');
  }
  istirahat(): void {
    console.log('Robot idle...');
  }
  makan(): void {
    throw new Error('Robot tidak makan!');
  }
}
```

✅ **Sesudah (ISP)**

```ts
interface Workable {
  kerja(): void;
}

interface Restable {
  istirahat(): void;
}

interface Eatable {
  makan(): void;
}

class HumanWorker implements Workable, Restable, Eatable {
  kerja(): void {
    console.log('Manusia bekerja...');
  }
  istirahat(): void {
    console.log('Manusia istirahat...');
  }
  makan(): void {
    console.log('Manusia makan...');
  }
}
```

```
class RobotWorkerISP implements Workable, Restable {
  kerja(): void {
    console.log('Robot bekerja...');
  }
  istirahat(): void {
    console.log('Robot idle...');
  }
  // □ Nggak perlu implement makan()
}
```

Pakai ISP kalau: Ada interface besar dengan banyak method tapi beberapa implementasinya nggak butuh semuanya.

D — Dependency Inversion Principle (DIP)

> **Class tingkat tinggi jangan bergantung langsung ke class tingkat rendah. Dua-duanya harus bergantung ke abstraksi (interface).**

□ Sebelum (langgar DIP)

```
// □ Langsung panggil class konkret
class MySQLDatabase {
  simpan(data: string): void {
    console.log(`MySQL: Simpan ${data}`);
  }
}

class UserService {
  private db: MySQLDatabase;

  constructor() {
    this.db = new MySQLDatabase(); // Kaku — ganti PostgreSQL susah
  }

  simpanUser(data: string): void {
    this.db.simpan(data);
  }
}
```

✅ Sesudah (DIP)

```typescript
interface Database {
  simpan(data: string): void;
}

class MySQLDatabaseDIP implements Database {
  simpan(data: string): void {
    console.log(`MySQL: Simpan ${data}`);
  }
}

class PostgreSQLDatabase implements Database {
  simpan(data: string): void {
    console.log(`PostgreSQL: Simpan ${data}`);
  }
}

class MongoDBDatabase implements Database {
  simpan(data: string): void {
    console.log(`MongoDB: Simpan ${data}`);
  }
}

class UserServiceDIP {
  constructor(private db: Database) {} // ✅ Bergantung ke abstraksi

  simpanUser(data: string): void {
    this.db.simpan(data);
  }
}

// Ganti DB tinggal ganti parameter:
const service = new UserServiceDIP(new MongoDBDatabase());
service.simpanUser('Budi');
```

Pakai DIP kalau: Kode kamu terikat ke detail teknis (database, API eksternal, file system) yang mungkin berubah.

Latihan

Soal 1 — SRP

Class berikut langgar SRP. Refactor jadi beberapa class dengan tanggung jawab terpisah.

```
class OrderProcessor {
  process(order: any): void {
    // Validasi
    if (!order.items || order.items.length === 0) {
      throw new Error('Order kosong');
    }
    // Hitung total
    let total = 0;
    for (const item of order.items) {
      total += item.price * item.qty;
    }
    // Simpan
    console.log(`Simpan order ${order.id} — total Rp${total}`);
    // Kirim invoice
    console.log(`Kirim invoice ke ${order.email}`);
  }
}
```

Soal 2 — OCP

Buat sistem diskon yang OCP-friendly. Saat ini cuma diskon member 10%. Nanti bakal nambah diskon VIP 20%, lebaran 15%, dan banyak lagi.

```
// Implementasi awal — tolong refactor
function hitungDiskon(harga: number, tipe: string): number {
  if (tipe === 'member') return harga * 0.9;
  return harga;
}
```

Soal 3 — ISP

Interface `MediaPlayer` punya method `playAudio()`, `playVideo()`, `showLyrics()`. Ada class `AudioPlayer` yang cuma butuh `playAudio()`. Refactor pake ISP.

Soal 4 — DIP

Service berikut langsung panggil `FirebaseAuth` konkret. Refactor pake DIP biar bisa ganti auth provider (Auth0, Supabase, custom).

```
class FirebaseAuth {
  login(email: string, password: string): Promise<string> {
    return Promise.resolve(`firebase-token-${email}`);
  }
}
```

```
class AuthService {
  private auth = new FirebaseAuth();

  async loginUser(email: string, password: string): Promise<string> {
    return this.auth.login(email, password);
  }
}
```

Selamat mengerjakan! SOLID bukan aturan saklek — tapi pedoman biar kode lebih sehat.

Sesi 2 — Creational Patterns

> **Creational patterns** ngurusin cara bikin objek. Biar pembuatan objek fleksibel, nggak kaku, dan nggak nyebarin new di mana-mana.

---

1. Singleton

> **Mastikan suatu class cuma punya satu instance sepanjang aplikasi.**

Kapan dipakai

- Koneksi database (biar nggak boros koneksi)
- Logger global
- Konfigurasi aplikasi (config object yang sama di semua module)
- Cache service

Contoh: DB Connection Singleton

```
class DatabaseConnection {
  private static instance: DatabaseConnection;
  private connected = false;

  private constructor() {} // 🔒 Constructor private — nggak bisa new dari luar

  static getInstance(): DatabaseConnection {
    if (!DatabaseConnection.instance) {
      DatabaseConnection.instance = new DatabaseConnection();
```

```
    }
    return DatabaseConnection.instance;
  }

  connect(): void {
    if (!this.connected) {
      console.log('🔌 Koneksi ke database...');
      this.connected = true;
      console.log('✅ Database terhubung');
    } else {
      console.log('♻️ Pakai koneksi yang udah ada');
    }
  }

  query(sql: string): void {
    if (!this.connected) {
      throw new Error('Belum connect ke DB!');
    }
    console.log(`📝 Execute: ${sql}`);
  }
}

// Pemakaian
const db1 = DatabaseConnection.getInstance();
const db2 = DatabaseConnection.getInstance();

db1.connect(); // 🔌 Koneksi ke database... ✅ Database terhubung
db2.connect(); // ♻️ Pakai koneksi yang udah ada

console.log(db1 === db2); // true — instance SAMA
```

Singleton di Express (App Config)

```
class AppConfig {
  private static instance: AppConfig;

  public readonly port: number;
  public readonly dbUrl: string;
  public readonly jwtSecret: string;
  public readonly environment: string;

  private constructor() {
    // Baca dari environment variable, fallback ke default
    this.port = Number(process.env.PORT) || 3000;
    this.dbUrl = process.env.DATABASE_URL || 'postgres://localhost:5432/myapp';
```

```
    this.jwtSecret = process.env.JWT_SECRET || 'default-secret';
    this.environment = process.env.NODE_ENV || 'development';
  }

  static getInstance(): AppConfig {
    if (!AppConfig.instance) {
      AppConfig.instance = new AppConfig();
    }
    return AppConfig.instance;
  }

  isProduction(): boolean {
    return this.environment === 'production';
  }
}

// Di mana aja, panggil:
// const config = AppConfig.getInstance();
```

Singleton di Mastra (Logger Service)

```
class LoggerService {
  private static instance: LoggerService;

  private constructor() {}

  static getInstance(): LoggerService {
    if (!LoggerService.instance) {
      LoggerService.instance = new LoggerService();
    }
    return LoggerService.instance;
  }

  info(message: string, data?: any): void {
    console.log(`[INFO] ${message}`, data ?? '');
  }

  error(message: string, error?: any): void {
    console.error(`[ERROR] ${message}`, error ?? '');
  }

  warn(message: string): void {
    console.warn(`[WARN] ${message}`);
  }
}
```

```
// // Contoh Mastra tool pake logger singleton
// import { MastraTool } from '@mastra/core';
//
// export const processOrderTool = new MastraTool({
//   name: 'processOrder',
//   execute: async ({ data }) => {
//     const log = LoggerService.getInstance();
//     log.info('Processing order', data);
//     // ... logic
//     log.info('Order processed');
//     return { success: true };
//   },
// });
```

Catatan: Singleton sering dikritik karena bikin global state. Di TypeScript modern, sering diganti **dependency injection** atau **module-level instance** (export instance dari module — lebih aman).

---

2. Factory

> **Sediakan cara bikin objek tanpa sebut class spesifiknya. Class yang beda-beda bisa muncul dari satu fungsi factory.**

Kapan dipakai

- Jenis objek ditentukan runtime (berdasarkan input user/config)
- Pembuatan objek rumit (banyak setup)
- Mau nyembunihin logika instantiasi dari consumer

Contoh: Factory untuk Notifikasi

```
interface Notifikasi {
  kirim(pesan: string, tujuan: string): void;
}

class EmailNotifikasi implements Notifikasi {
  kirim(pesan: string, tujuan: string): void {
    console.log(`□ Email ke ${tujuan}: ${pesan}`);
  }
}

class SMSNotifikasi implements Notifikasi {
```

```
  kirim(pesan: string, tujuan: string): void {
    console.log(`□ SMS ke ${tujuan}: ${pesan}`);
  }
}

class WhatsAppNotifikasi implements Notifikasi {
  kirim(pesan: string, tujuan: string): void {
    console.log(`□ WhatsApp ke ${tujuan}: ${pesan}`);
  }
}

// □ Factory function
function buatNotifikasi(jenis: string): Notifikasi {
  switch (jenis) {
    case 'email':
      return new EmailNotifikasi();
    case 'sms':
      return new SMSNotifikasi();
    case 'wa':
      return new WhatsAppNotifikasi();
    default:
      throw new Error(`Tipe notifikasi '${jenis}' nggak dikenal`);
  }
}

// Pemakaian
const notif = buatNotifikasi('email');
notif.kirim('Pesanan kamu sudah dikirim!', 'budi@email.com');
```

Factory di Express (Bikin Response)

```
interface ApiResponse<T> {
  success: boolean;
  data: T;
  message: string;
  timestamp: Date;
}

class ApiResponseFactory {
  static success<T>(data: T, message = 'OK'): ApiResponse<T> {
    return {
      success: true,
      data,
      message,
      timestamp: new Date(),
```

```
    };
  }

  static error(message: string, code?: number): ApiResponse<null> {
    return {
      success: false,
      data: null,
      message: `Error ${code ?? 500}: ${message}`,
      timestamp: new Date(),
    };
  }
}

// Router express
// router.get('/users', async (req, res) => {
//   try {
//     const users = await userRepo.findAll();
//     res.json(ApiResponseFactory.success(users));
//   } catch (err) {
//     res.status(500).json(ApiResponseFactory.error('Gagal ambil users'));
//   }
// });
```

Factory di Mastra (Dynamic Agent)

```
// Bayangin kita punya beberapa tipe Mastra agent
interface AgentConfig {
  name: string;
  model: string;
  instructions: string;
}

function createAgent(jenis: string): AgentConfig {
  switch (jenis) {
    case 'customer-support':
      return {
        name: 'customer-support-agent',
        model: 'gpt-4',
        instructions: 'Bantu customer dengan pertanyaan produk dan pesanan.',
      };
    case 'data-analyst':
      return {
        name: 'data-analyst-agent',
        model: 'gpt-4-turbo',
        instructions: 'Analisis data dan buat laporan.',
```

```
      };
    case 'translator':
      return {
        name: 'translator-agent',
        model: 'gpt-4',
        instructions: 'Terjemahkan teks ke bahasa Indonesia.',
      };
    default:
      throw new Error(`Unknown agent type: ${jenis}`);
  }
}
```

---

3. Builder

> **Pisahin pembuatan objek kompleks jadi langkah-langkah kecil. Biar nggak bikin constructor raksasa.**

Kapan dipakai

- Objek punya banyak parameter opsional (> 4 parameter)
- Ada objek yang butuh konfigurasi bertahap
- Mau объект di-create dengan berbagai variasi konfigurasi tanpa bikin constructor overload

Contoh: Builder untuk HTTP Request

```
class HttpRequest {
  constructor(
    public readonly method: string,
    public readonly url: string,
    public readonly headers: Record<string, string>,
    public readonly body?: any,
    public readonly timeout?: number,
    public readonly retry?: number,
    public readonly cache?: boolean
  ) {}
}

class HttpRequestBuilder {
  private method = 'GET';
  private url = '';
  private headers: Record<string, string> = {};
  private body?: any;
```

```
private timeout = 5000;
private retry = 0;
private cache = false;

setMethod(method: string): this {
  this.method = method.toUpperCase();
  return this;
}

setUrl(url: string): this {
  this.url = url;
  return this;
}

addHeader(key: string, value: string): this {
  this.headers[key] = value;
  return this;
}

setBody(body: any): this {
  this.body = body;
  return this;
}

setTimeout(timeout: number): this {
  this.timeout = timeout;
  return this;
}

setRetry(retry: number): this {
  this.retry = retry;
  return this;
}

enableCache(): this {
  this.cache = true;
  return this;
}

build(): HttpRequest {
  if (!this.url) {
    throw new Error('URL wajib diisi!');
  }
  return new HttpRequest(
    this.method,
    this.url,
```

```
      this.headers,
      this.body,
      this.timeout,
      this.retry,
      this.cache
    );
  }
}

// Pemakaian — jelas dan terbaca
const request = new HttpRequestBuilder()
  .setMethod('POST')
  .setUrl('https://api.example.com/orders')
  .addHeader('Authorization', 'Bearer token123')
  .addHeader('Content-Type', 'application/json')
  .setBody({ userId: 1, items: ['item-1', 'item-2'] })
  .setTimeout(10000)
  .setRetry(3)
  .enableCache()
  .build();

console.log(request);
```

Builder di Express (Route Config)

```
interface RouteOptions {
  path: string;
  method: 'GET' | 'POST' | 'PUT' | 'DELETE';
  middlewares: any[];
  handler: any;
  rateLimit?: number;
  cache?: boolean;
}

class RouteConfigurator {
  private route: Partial<RouteOptions> = {};

  path(p: string): this {
    this.route.path = p;
    return this;
  }

  method(m: RouteOptions['method']): this {
    this.route.method = m;
    return this;
```

```
  }

  middlewares(...mw: any[]): this {
    this.route.middlewares = mw;
    return this;
  }

  handler(h: any): this {
    this.route.handler = h;
    return this;
  }

  rateLimit(messagesPerMinute: number): this {
    this.route.rateLimit = messagesPerMinute;
    return this;
  }

  withCache(): this {
    this.route.cache = true;
    return this;
  }

  build(): RouteOptions {
    if (!this.route.path || !this.route.method || !this.route.handler) {
      throw new Error('path, method, handler wajib diisi');
    }
    return this.route as RouteOptions;
  }
}

// // Pemakaian di router express
// const route = new RouteConfigurator()
//   .path('/users')
//   .method('GET')
//   .middlewares(authMiddleware, rateLimiter)
//   .handler(getUsersHandler)
//   .withCache()
//   .build();
//
// router[route.method](route.path, ...route.middlewares, route.handler);
```

Builder di Mastra (Tool Builder)

```
class MastraToolBuilder {
  private config: any = {};
```

```
  name(n: string): this {
    this.config.name = n;
    return this;
  }

  description(d: string): this {
    this.config.description = d;
    return this;
  }

  schema(s: any): this {
    this.config.schema = s;
    return this;
  }

  execute(fn: (args: any) => Promise<any>): this {
    this.config.execute = fn;
    return this;
  }

  retryPolicy(maxRetries: number, delayMs: number): this {
    this.config.retryPolicy = { maxRetries, delayMs };
    return this;
  }

  cacheTTL(ms: number): this {
    this.config.cacheTTL = ms;
    return this;
  }

  build() {
    // @ts-ignore — Mastra tool constructor
    return new MastraTool(this.config);
  }
}

// // Pemakaian
// const searchTool = new MastraToolBuilder()
//   .name('searchProduct')
//   .description('Cari produk berdasarkan keyword')
//   .schema(z.object({ keyword: z.string() }))
//   .execute(async ({ keyword }) => {
//     return await productRepo.search(keyword);
//   })
//   .retryPolicy(3, 1000)
```

```
//   .build();
```

Ringkasan

Pattern	Gampangnya	Kapan Pake
Singleton	Cuma satu instance sepanjang app	Koneksi DB, config global, logger
Factory	Bikin objek tanpa sebut class spesifik	Jenis objek ditentukan runtime
Builder	Bikin objek step-by-step	Constructor penuh parameter opsional

---

Latihan

Soal 1 — Singleton

Buat class `CacheService` singleton yang bisa nyimpen data di memory dengan method `get(key)`, `set(key, value)`, `clear()`. Pastikan cuma ada satu instance di seluruh aplikasi.

Soal 2 — Factory

Kamu punya beberapa metode pembayaran: `BankTransfer`, `Ewallet`, `CreditCard`. Masing-masing punya method `bayar(jumlah: number): string`. Buat factory `PaymentFactory.buat(jenis: string): Pembayaran` yang return class sesuai jenis.

Contoh:

```
const pembayaran = PaymentFactory.buat('gopay');
console.log(pembayaran.bayar(50000)); // "Bayar Rp50000 via GoPay"
```

Soal 3 — Builder

Buat class `EmailMessage` dengan properti: `to`, `cc`, `bcc`, `subject`, `body`, `attachments[]`, `priority` (low/normal/high). Terus bikin `EmailBuilder` yang bisa di-chain.

```
const email = new EmailBuilder()
  .to('budi@email.com')
  .subject('Meeting Reminder')
  .body('Jangan lupa meeting jam 10')
  .addAttachment('meeting.pdf')
  .priority('high')
  .build();
```

Soal 4 — Terapkan ke Express

Refactor route berikut pake Factory + Builder:

```
app.get('/products', async (req, res) => {
  try {
    const products = await db.query('SELECT * FROM products');
    res.json({ success: true, data: products });
  } catch (err) {
    res.status(500).json({ success: false, message: 'Error' });
  }
});

app.post('/products', async (req, res) => {
  try {
    await db.query('INSERT INTO products ...');
    res.json({ success: true, message: 'Product created' });
  } catch (err) {
    res.status(500).json({ success: false, message: 'Error' });
  }
});
```

Selamat mencoba! Pattern ini alat, bukan tujuan. Pake kalau memang ada masalah yang cocok.

Sesi 3 — Structural & Behavioral Patterns

> **Structural patterns** ngatur hubungan antar class/objek biar strukturnya fleksibel.
> **Behavioral patterns** ngatur komunikasi antar objek — gimana mereka saling kirim pesan dan koordinasi.

Bagian 1 — Structural Patterns

---

1. Adapter

> **Jembatin dua class yang interface-nya beda biar bisa kerja sama. Kayak adaptor colokan listrik.**

Kapan dipakai

- Integrasi library lama dengan code baru
- API eksternal yang formatnya nggak cocok
- Mau pake class yang udah ada tapi method-nya beda

Contoh: Adapter untuk Library Pembayaran

```
// Library lama — format beda
class OldPaymentGateway {
  processPayment(amount: number, currency: string): string {
    return `[OLD] Payment ${amount} ${currency} processed.`;
  }
}

// Interface baru yang kita mau pake
interface PaymentGateway {
  pay(amount: number): string;
}

// Adapter — jembatin old ke new
class PaymentAdapter implements PaymentGateway {
  constructor(private oldGateway: OldPaymentGateway) {}

  pay(amount: number): string {
    // Adapter nyembunyiin detail konversi
    return this.oldGateway.processPayment(amount, 'IDR');
  }
}

// Pemakaian
const oldGateway = new OldPaymentGateway();
const adapter = new PaymentAdapter(oldGateway);
console.log(adapter.pay(50000)); // "[OLD] Payment 50000 IDR processed."

// Kalau nanti ganti Stripe, tinggal bikin adapter baru
class StripePayment implements PaymentGateway {
  pay(amount: number): string {
    return `[Stripe] Charged $${(amount / 15000).toFixed(2)}`;
  }
}
```

---

2. Decorator

> **Nambahin perilaku ke objek secara dinamis tanpa ngubah class asli. Kayak topping es krim.**

Kapan dipakai

- Middleware di Express (log, auth, rate-limit)
- Nambah fitur ke objek tanpa inheritance
- Banyak kombinasi fitur yang bisa dipilih-pilih

Contoh: Decorator sebagai Middleware Pattern

```ts
// Interface handler
interface RequestHandler {
  handle(req: any): string;
}

// Handler dasar
class BaseHandler implements RequestHandler {
  handle(req: any): string {
    return `Handling: ${req.url}`;
  }
}

// Decorator base
abstract class HandlerDecorator implements RequestHandler {
  constructor(protected handler: RequestHandler) {}

  abstract handle(req: any): string;
}

// Decorator: Logging
class LoggingDecorator extends HandlerDecorator {
  handle(req: any): string {
    console.log(`[LOG] ${req.method} ${req.url} — ${new Date().toISOString()}`);
    return this.handler.handle(req);
  }
}

// Decorator: Auth
class AuthDecorator extends HandlerDecorator {
  handle(req: any): string {
    if (!req.headers?.authorization) {
      throw new Error('Unauthorized!');
    }
```

```
    console.log('[AUTH] Token valid');
    return this.handler.handle(req);
  }
}

// Decorator: Rate Limiting
class RateLimitDecorator extends HandlerDecorator {
  private requestCount = 0;
  private readonly maxRequests = 100;

  handle(req: any): string {
    this.requestCount++;
    if (this.requestCount > this.maxRequests) {
      throw new Error('Rate limit exceeded!');
    }
    console.log(`[RATE] Request ${this.requestCount}/${this.maxRequests}`);
    return this.handler.handle(req);
  }
}

// Pemakaian — stack decorator kayak middleware Express
let handler: RequestHandler = new BaseHandler();
handler = new LoggingDecorator(handler);
handler = new AuthDecorator(handler);
handler = new RateLimitDecorator(handler);

handler.handle({
  method: 'GET',
  url: '/users',
  headers: { authorization: 'Bearer token123' },
});
// [LOG] GET /users — 2025-01-15T10:30:00.000Z
// [AUTH] Token valid
// [RATE] Request 1/100
// Handling: /users
```

Decorator di Express — Middleware Nyata

```
import { Request, Response, NextFunction } from 'express';

function logger(req: Request, _res: Response, next: NextFunction): void {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
}
```

```ts
function auth(req: Request, res: Response, next: NextFunction): void {
  if (!req.headers.authorization) {
    res.status(401).json({ error: 'Unauthorized' });
    return;
  }
  next();
}

function rateLimit(max: number) {
  const counts = new Map<string, number>();
  return (req: Request, res: Response, next: NextFunction): void => {
    const ip = req.ip ?? 'unknown';
    const current = (counts.get(ip) ?? 0) + 1;
    counts.set(ip, current);
    if (current > max) {
      res.status(429).json({ error: 'Too many requests' });
      return;
    }
    next();
  };
}

// // Route pake decorator (middleware)
// app.get('/users', logger, auth, rateLimit(100), (req, res) => {
//   res.json({ users: [] });
// });
```

---

3. Proxy

> **Sediain pengganti atau perantara buat ngontrol akses ke objek asli.**

Kapan dipakai

- Caching (nyimpen hasil query biar nggak hit DB terus)
- Access control (cek role sebelum akses)
- Lazy loading (bikin objek berat cuma kalau dipake)

Contoh: Proxy untuk Caching

```ts
interface UserRepository {
  findById(id: number): Promise<any>;
}
```

```typescript
// Implementasi asli — akses database
class DatabaseUserRepository implements UserRepository {
  async findById(id: number): Promise<any> {
    console.log(`🗄️ Query DB: SELECT * FROM users WHERE id = ${id}`);
    // Simulasi delay query
    await new Promise((r) => setTimeout(r, 1000));
    return { id, name: `User ${id}`, email: `user${id}@email.com` };
  }
}

// Proxy — tambahin cache di depan DB
class CachedUserRepository implements UserRepository {
  private cache = new Map<number, any>();

  constructor(private db: UserRepository) {}

  async findById(id: number): Promise<any> {
    if (this.cache.has(id)) {
      console.log(`⚡ Cache HIT: user ${id}`);
      return this.cache.get(id);
    }

    console.log(`🗄️ Cache MISS: user ${id}`);
    const user = await this.db.findById(id);
    this.cache.set(id, user);
    return user;
  }
}

// Pemakaian
async function main() {
  const db = new DatabaseUserRepository();
  const cache = new CachedUserRepository(db);

  console.log('=== Request 1 ===');
  await cache.findById(1); // 🗄️ Query DB + 🗄️ Cache MISS

  console.log('\n=== Request 2 ===');
  await cache.findById(1); // ⚡ Cache HIT — nggak query DB!

  console.log('\n=== Request 3 ===');
  await cache.findById(2); // 🗄️ Query DB (user 2 belum di-cache)
}

main();
```

Proxy di Express (Guard/Auth Middleware)

```
// Proxy pattern di Express ya... middleware! Setiap middleware
// bisa ngecek, nge-log, atau nge-cache sebelum request sampe ke handler asli.

// Contoh: Proxy untuk guard role
function roleGuard(...allowedRoles: string[]) {
  return (req: any, res: any, next: any): void => {
    const userRole = req.user?.role;
    if (!userRole || !allowedRoles.includes(userRole)) {
      res.status(403).json({ error: 'Forbidden' });
      return;
    }
    next();
  };
}

// // Route
// app.delete('/users/:id', auth, roleGuard('admin'), deleteUserHandler);
```

Bagian 2 — Behavioral Patterns

---

4. Observer

> **Satu objek (subject) ngasih tau banyak objek lain (observer) kalau ada perubahan. Kayak subscribe YouTube.**

Kapan dipakai

- Event system / EventEmitter
- UI updates (komponen A berubah, komponen B & C ikut update)
- Real-time data (WebSocket notification)

Contoh: Observer dengan EventEmitter

```
// Simple EventEmitter dari awal
type EventHandler = (...args: any[]) => void;

class EventEmitter {
  private events = new Map<string, EventHandler[]>();
```

```
  on(event: string, handler: EventHandler): void {
    const handlers = this.events.get(event) ?? [];
    handlers.push(handler);
    this.events.set(event, handlers);
  }

  off(event: string, handler: EventHandler): void {
    const handlers = this.events.get(event);
    if (!handlers) return;
    this.events.set(
      event,
      handlers.filter((h) => h !== handler)
    );
  }

  emit(event: string, ...args: any[]): void {
    const handlers = this.events.get(event);
    if (!handlers) return;
    handlers.forEach((handler) => handler(...args));
  }
}

// Pake EventEmitter buat sistem notifikasi
const eventBus = new EventEmitter();

// Observer 1 — Email Service
eventBus.on('order:created', (order: any) => {
  console.log(`□ Email: Pesanan ${order.id} berhasil dibuat.`);
});

// Observer 2 — SMS Service
eventBus.on('order:created', (order: any) => {
  console.log(`□ SMS: Pesanan ${order.id} diterima.`);
});

// Observer 3 — Analytics Service
eventBus.on('order:created', (order: any) => {
  console.log(`□ Analytics: Catat order ${order.id} — Rp${order.total}`);
});

// Trigger — satu emit, tiga observer jalan
eventBus.emit('order:created', { id: 'ORD-001', total: 150000 });
// □ Email: Pesanan ORD-001 berhasil dibuat.
// □ SMS: Pesanan ORD-001 diterima.
// □ Analytics: Catat order ORD-001 — Rp150000
```

Observer di Express (Event Bus untuk Webhook)

```
// // Event bus global — semua modul bisa subscribe
// const eventBus = new EventEmitter();
//
// // Subscribe di module email
// eventBus.on('payment:success', async (data) => {
//   await emailService.sendInvoice(data.email, data.invoiceUrl);
// });
//
// // Subscribe di module inventory
// eventBus.on('payment:success', async (data) => {
//   await inventoryService.reduceStock(data.items);
// });
//
// // Route payment — emit event
// app.post('/payments/callback', async (req, res) => {
//   const result = await paymentService.process(req.body);
//   eventBus.emit('payment:success', result);
//   res.json({ status: 'ok' });
// });
```

---

5. Strategy

> **Definin grup algoritma yang bisa ditukar-tukar run-time. Keluarin logika yang berubah-ubah ke class strategy sendiri.**

Kapan dipakai

- Banyak cara/cabang logika dalam satu proses (pembayaran, diskon, shipping)
- Mau ganti algoritma runtime tanpa if-else panjang
- Testing jadi gampang — tinggal pake mock strategy

Contoh: Strategy untuk Shipping Cost

```
interface ShippingStrategy {
  hitung(beratKg: number): number;
}

class RegularShipping implements ShippingStrategy {
  hitung(beratKg: number): number {
    return beratKg * 5000; // Rp5000/kg
```

```
  }
}

class ExpressShipping implements ShippingStrategy {
  hitung(beratKg: number): number {
    return beratKg * 10000 + 15000; // Rp10000/kg + flat Rp15000
  }
}

class SameDayShipping implements ShippingStrategy {
  hitung(beratKg: number): number {
    return beratKg * 20000 + 25000; // Mahal tapi cepet
  }
}

class FreeShipping implements ShippingStrategy {
  hitung(_beratKg: number): number {
    return 0; // Gratis!
  }
}

// Context — pake strategy
class ShippingCalculator {
  constructor(private strategy: ShippingStrategy) {}

  setStrategy(strategy: ShippingStrategy): void {
    this.strategy = strategy;
  }

  calculate(beratKg: number): number {
    const cost = this.strategy.hitung(beratKg);
    console.log(`□ ${this.strategy.constructor.name}: Rp${cost.toLocaleString('i
    return cost;
  }
}

// User bisa milih pengiriman
const calculator = new ShippingCalculator(new RegularShipping());
calculator.calculate(2); // Rp10.000

calculator.setStrategy(new ExpressShipping());
calculator.calculate(2); // Rp35.000

calculator.setStrategy(new FreeShipping());
calculator.calculate(2); // Rp0
```

Strategy di Express (Dynamic Response Format)

```typescript
interface ResponseStrategy {
  format(data: any, message: string): any;
}

class JSONResponse implements ResponseStrategy {
  format(data: any, message: string): any {
    return { success: true, message, data, timestamp: new Date() };
  }
}

class XMLResponse implements ResponseStrategy {
  format(data: any, message: string): string {
    let xml = `<?xml version="1.0"?>\n<response>\n`;
    xml += `  <message>${message}</message>\n`;
    xml += `  <data>${JSON.stringify(data)}</data>\n`;
    xml += `</response>`;
    return xml;
  }
}

class ResponseContext {
  constructor(private strategy: ResponseStrategy) {}

  setStrategy(strategy: ResponseStrategy): void {
    this.strategy = strategy;
  }

  send(res: any, data: any, message = 'OK'): void {
    const formatted = this.strategy.format(data, message);
    res.json(formatted);
  }
}

// // Route express
// const responseContext = new ResponseContext(new JSONResponse());
//
// app.get('/users', async (req, res) => {
//   // Kalau header Accept: application/xml, ganti strategy
//   if (req.headers.accept === 'application/xml') {
//     responseContext.setStrategy(new XMLResponse());
//   }
//   const users = await userRepo.findAll();
//   responseContext.send(res, users);
// });
```

Strategy di Mastra (Dynamic Tool Behavior)

```ts
// Strategy buat nentuin gimana response diformat
interface ResponseFormatStrategy {
  format(raw: any): string;
}

class ConciseFormat implements ResponseFormatStrategy {
  format(raw: any): string {
    return `Hasil: ${JSON.stringify(raw)}`;
  }
}

class DetailedFormat implements ResponseFormatStrategy {
  format(raw: any): string {
    return `Detail Lengkap:\n${JSON.stringify(raw, null, 2)}`;
  }
}

// // Mastra tool pake strategy
// const searchTool = new MastraTool({
//   name: 'searchWithFormat',
//   execute: async ({ data }) => {
//     const format = data.detailed
//       ? new DetailedFormat()
//       : new ConciseFormat();
//     const results = await searchRepo.search(data.query);
//     return format.format(results);
//   },
// });
```

Ringkasan

Pattern	Gampangnya	Kapan Pake
Adapter	Jembatin interface beda	Integrasi library lama/pihak ketiga
Decorator	Nambah fitur tanpa ubah class asli	Middleware, kombinasi fitur
Proxy	Perantara buat kontrol akses	Caching, guard, lazy loading
Observer	Satu ngasih tau banyak	Event system, notifikasi, real-time
Strategy	Tukar algoritma runtime	Banyak varian algoritma dalam satu proses

---

Latihan

Soal 1 — Adapter

Kamu pake library `sendEmail(from, to, subject, body)` (format lama). Interface baru yang kamu mau pake adalah `EmailService.send(EmailPayload)`. Buat adapter-nya.

```
// Library lama
class OldMailer {
  sendEmail(from: string, to: string, subject: string, body: string): string {
    return `Email sent from ${from} to ${to}`;
  }
}

// Interface baru
interface EmailService {
  send(payload: { from: string; to: string; subject: string; body: string }): st
}
```

Soal 2 — Decorator

Buat decorator `CompressionDecorator` dan `EncryptionDecorator` yang bisa stack ke `DataService`:

```
interface DataService {
  write(data: string): void;
}

class FileDataService implements DataService {
  write(data: string): void {
    console.log(`Write to file: ${data}`);
  }
}

// Bikin decorator-nya!
```

Soal 3 — Proxy

Buat proxy `RateLimitedProxy` yang ngebatasi akses ke `ApiService` maksimal 5 request per menit per user.

```
class ApiService {
  fetch(endpoint: string): string {
    return `Data dari ${endpoint}`;
  }
}
```

```
// Proxy di sini
```

Soal 4 — Observer + Strategy

Skenario: Ada sistem **order created**. Waktu order dibuat, dua hal terjadi:

1. Observer pattern: Notifikasi (email + SMS)
2. Strategy pattern: Diskon beda-beda tergantung tipe member

Buat implementasi lengkapnya.

Latihan ini real-world banget — pas kerja nanti kamu bakal nemu kombinasi pattern kayak gini setiap hari.

Sesi 4 — Functional Programming & Real-World Patterns

> **Functional programming** treat function sebagai ”warga negara kelas satu”. Code lebih prediktabel, gampang di-test, dan jarang error misterius.

---

1. Pure Function

> **Fungsi murni: input sama → output sama. Nggak ada efek samping. Nggak ubah variable global.**

❌ Impure Function

```
let totalBelanja = 0;

function tambahKeKeranjang(harga: number): number {
  totalBelanja += harga; // ❌ Mutasi global!
  return totalBelanja;
}

console.log(tambahKeKeranjang(10000)); // 10000
console.log(tambahKeKeranjang(10000)); // 20000 — hasil beda meski input sama!
```

✅ Pure Function

```ts
function hitungTotal(keranjang: number[], itemBaru: number): number {
  return [...keranjang, itemBaru].reduce((a, b) => a + b, 0);
}

const keranjang = [10000];
console.log(hitungTotal(keranjang, 10000)); // 20000
console.log(hitungTotal(keranjang, 10000)); // 20000 — selalu sama!
console.log(keranjang); // [10000] — array asli nggak berubah
```

Keuntungan pure function:

- Gampang di-test (nggak perlu setup state global)
- Hasil konsisten
- Bisa di-cache (memoization)
- Thread-safe

---

2. Immutability

> **Data nggak boleh diubah setelah dibuat. Kalau mau "ubah", bikin salinan baru.**

❌ Mutasi

```ts
const user = { name: 'Budi', address: { city: 'Jakarta' } };

function updateCity(user: any, newCity: string) {
  user.address.city = newCity; // ❌ Mutasi nested object!
  return user;
}

const updated = updateCity(user, 'Bandung');
console.log(user.city); // 'Bandung' — object asli berubah! ❌
```

✅ Immutable

```ts
function updateCityImmutable(user: any, newCity: string) {
  return {
    ...user,
    address: {
      ...user.address,
      city: newCity, // Object asli aman
    },
  };
```

```
}

const user2 = { name: 'Budi', address: { city: 'Jakarta' } };
const updated2 = updateCityImmutable(user2, 'Bandung');

console.log(user2.address.city); // 'Jakarta' — aman
console.log(updated2.address.city); // 'Bandung'
```

Immutability di Array

```
const todos = ['Belajar TS', 'Bikin API'];

// □ Mutasi
todos.push('Belajar React'); // ngerusak array asli

// □ Immutable
const newTodos = [...todos, 'Belajar React'];
const withoutFirst = todos.slice(1);
const updatedTodos = todos.map((t, i) => (i === 0 ? `${t} □` : t));
```

---

3. Function Composition

> **Gabungin beberapa fungsi kecil jadi satu fungsi kompleks. Kayak pipa di pabrik.**

```
// Fungsi kecil — masing-masing pure
const trim = (s: string): string => s.trim();
const lower = (s: string): string => s.toLowerCase();
const slugify = (s: string): string => s.replace(/\s+/g, '-');
const truncate = (max: number) => (s: string): string =>
  s.length > max ? s.slice(0, max) + '...' : s;

// Komposisi manual
function createSlug(title: string): string {
  return slugify(lower(trim(title)));
}

console.log(createSlug('  Hello World  ')); // "hello-world"

// Generic compose helper
function compose<T>(...fns: ((arg: T) => T)[]): (arg: T) => T {
  return (arg: T) => fns.reduceRight((acc, fn) => fn(acc), arg);
}
```

```typescript
// Pipe — compose dari kiri ke kanan
function pipe<T>(...fns: ((arg: T) => T)[]): (arg: T) => T {
  return (arg: T) => fns.reduce((acc, fn) => fn(acc), arg);
}

const createNiceSlug = pipe(trim, lower, slugify, truncate(20));
console.log(createNiceSlug('  Artikel Tentang TypeScript  ')); // "artikel-tenta
```

---

4. Currying

> **Ubah fungsi dengan banyak parameter jadi rantai fungsi satu parameter.**

Kenapa currying?

- Bikin **partial application** — set beberapa parameter dulu, sisanya belakangan
- Fungsi jadi reusable dengan konfigurasi berbeda

```typescript
// ❌ Biasa
function formatCurrency(amount: number, currency: string, locale: string): strin
  return new Intl.NumberFormat(locale, {
    style: 'currency',
    currency,
  }).format(amount);
}

formatCurrency(50000, 'IDR', 'id-ID'); // "Rp50.000"
formatCurrency(150000, 'IDR', 'id-ID'); // "Rp150.000"
// Repetisi: tiap manggil harus sebut 'IDR' dan 'id-ID'

// ✅ Currying
function formatCurrencyCurried(currency: string) {
  return function (locale: string) {
    return function (amount: number): string {
      return new Intl.NumberFormat(locale, {
        style: 'currency',
        currency,
      }).format(amount);
    };
  };
}

// Set konfigurasi sekali
```

```ts
const formatIDR = formatCurrencyCurried('IDR')('id-ID');
const formatUSD = formatCurrencyCurried('USD')('en-US');

console.log(formatIDR(50000)); // "Rp50.000"
console.log(formatIDR(150000)); // "Rp150.000"
console.log(formatUSD(50)); // "$50.00"

// Arrow function version
const curried =
  (currency: string) => (locale: string) => (amount: number) =>
    new Intl.NumberFormat(locale, { style: 'currency', currency }).format(amount
```

Currying di Express Middleware

```ts
// Middleware dengan parameter — pake currying
function requireRole(role: string) {
  return (req: any, res: any, next: any): void => {
    if (req.user?.role !== role) {
      res.status(403).json({ error: `Must be ${role}` });
      return;
    }
    next();
  };
}

function rateLimit(maxRequests: number, windowMs: number) {
  return (req: any, res: any, next: any): void => {
    // logic rate limiting
    next();
  };
}

// // Pake di route
// app.delete('/users/:id', requireRole('admin'), deleteHandler);
// app.get('/api/*', rateLimit(100, 60000), apiHandler);
```

---

5. Monads — Option & Result

> **Monad = wrapper yang ngurusin efek samping (null, error) biar code tetap pure.**

Option (Maybe) — handle null/undefined

```typescript
class Option<T> {
  private constructor(private value: T | null) {}

  static some<T>(value: T): Option<T> {
    return new Option(value);
  }

  static none<T>(): Option<T> {
    return new Option<T>(null);
  }

  isSome(): boolean {
    return this.value !== null;
  }

  isNone(): boolean {
    return this.value === null;
  }

  map<U>(fn: (value: T) => U): Option<U> {
    if (this.isNone()) return Option.none();
    return Option.some(fn(this.value!));
  }

  getOrElse(defaultValue: T): T {
    return this.value ?? defaultValue;
  }

  // Chain function yang return Option juga
  flatMap<U>(fn: (value: T) => Option<U>): Option<U> {
    if (this.isNone()) return Option.none();
    return fn(this.value!);
  }
}

// Pemakaian — tanpa null check bersarang
function findUser(id: number): Option<{ name: string; email: string }> {
  const users = [
    { id: 1, name: 'Budi', email: 'budi@email.com' },
  ];
  const user = users.find((u) => u.id === id);
  return user ? Option.some(user) : Option.none();
}
```

```ts
const userOption = findUser(1)
  .map((u) => u.name.toUpperCase())
  .getOrElse('USER NOT FOUND');

console.log(userOption); // "BUDI"

// Kalau user nggak ada — aman, nggak error
const userOption2 = findUser(999)
  .map((u) => u.name.toUpperCase())
  .getOrElse('USER NOT FOUND');

console.log(userOption2); // "USER NOT FOUND"
```

Result — handle error tanpa throw

```ts
class Result<T, E = Error> {
  private constructor(
    private readonly ok: boolean,
    private readonly value?: T,
    private readonly error?: E
  ) {}

  static success<T>(value: T): Result<T, never> {
    return new Result(true, value);
  }

  static failure<E>(error: E): Result<never, E> {
    return new Result(false, undefined, error);
  }

  isSuccess(): boolean {
    return this.ok;
  }

  isFailure(): boolean {
    return !this.ok;
  }

  getOrThrow(): T {
    if (this.isFailure()) throw this.error;
    return this.value!;
  }

  getOrElse(defaultValue: T): T {
    return this.isSuccess() ? this.value! : defaultValue;
```

```
  }

  map<U>(fn: (value: T) => U): Result<U, E> {
    if (this.isFailure()) return Result.failure(this.error!);
    return Result.success(fn(this.value!));
  }

  flatMap<U>(fn: (value: T) => Result<U, E>): Result<U, E> {
    if (this.isFailure()) return Result.failure(this.error!);
    return fn(this.value!);
  }
}

// Pemakaian — tanpa try/catch
function divide(a: number, b: number): Result<number, string> {
  if (b === 0) return Result.failure('Division by zero!');
  return Result.success(a / b);
}

const result = divide(10, 2)
  .map((n) => n * 2)
  .map((n) => `Hasil: ${n}`)
  .getOrElse('Ada error');

console.log(result); // "Hasil: 10"

const result2 = divide(10, 0)
  .map((n) => n * 2)
  .map((n) => `Hasil: ${n}`)
  .getOrElse('Ada error');

console.log(result2); // "Ada error" — nggak throw!
```

---

6. Applying Patterns ke Express Routes

Sebelum — Route biasa, campur aduk

```
import express from 'express';
const app = express();

app.get('/users/:id', async (req, res) => {
  try {
    const { id } = req.params;
    // Validasi
```

```
    if (!id || isNaN(Number(id))) {
      return res.status(400).json({ error: 'Invalid ID' });
    }
    // Query ke DB
    const user = await db.query('SELECT * FROM users WHERE id = $1', [id]);
    if (!user.rows[0]) {
      return res.status(404).json({ error: 'User not found' });
    }
    // Transform response
    const response = {
      id: user.rows[0].id,
      name: user.rows[0].name,
      email: user.rows[0].email,
      createdAt: user.rows[0].created_at,
    };
    res.json({ success: true, data: response });
  } catch (err) {
    res.status(500).json({ error: 'Internal server error' });
  }
});
```

✅ Sesudah — Refactor pake pattern

```
import express from 'express';
const app = express();

// --- Pure functions ---
const validateId = (id: string): Result<number, string> => {
  const num = Number(id);
  return isNaN(num) ? Result.failure('Invalid ID') : Result.success(num);
};

const toUserResponse = (row: any) => ({
  id: row.id,
  name: row.name,
  email: row.email,
  createdAt: row.created_at,
});

// --- Repository (DIP) ---
interface UserRepository {
  findById(id: number): Promise<any>;
}

class DBUserRepository implements UserRepository {
```

```
  async findById(id: number): Promise<any> {
    const result = await db.query('SELECT * FROM users WHERE id = $1', [id]);
    return result.rows[0] ?? null;
  }
}

// --- Service (strategy pattern) ---
class UserService {
  constructor(private repo: UserRepository) {}

  async getUser(id: string): Promise<Result<any, string>> {
    const idResult = validateId(id);
    if (idResult.isFailure()) return Result.failure(idResult.getOrElse(null));

    const user = await this.repo.findById(idResult.getOrThrow());
    if (!user) return Result.failure('User not found');

    return Result.success(toUserResponse(user));
  }
}

// --- Route handler dipisah (SRP) ---
function getUserHandler(userService: UserService) {
  return async (req: any, res: any): Promise<void> => {
    const result = await userService.getUser(req.params.id);

    if (result.isFailure()) {
      const error = result.getOrElse(null);
      const status = error === 'User not found' ? 404 : 400;
      res.status(status).json({ error });
      return;
    }

    res.json({ success: true, data: result.getOrThrow() });
  };
}

// --- Register route ---
const userRepo = new DBUserRepository();
const userService = new UserService(userRepo);
app.get('/users/:id', getUserHandler(userService));
```

---

7. Applying Patterns ke Mastra Tools

Sebelum — Tool mentah

```
// import { MastraTool } from '@mastra/core';

const rawTool = {
  name: 'getUserOrders',
  execute: async ({ data }: { data: { userId: string } }) => {
    // Langsung query di dalam tool
    const orders = await db.query('SELECT * FROM orders WHERE user_id = $1', [da

    if (!orders.rows) {
      throw new Error('Failed to fetch orders');
    }

    // Format manual
    return orders.rows.map((o: any) => ({
      id: o.id,
      total: o.total,
      status: o.status,
    }));
  },
};
```

✅ Sesudah — Refactor pake pattern

```
// --- Pure function: validasi ---
const validateUserId = (id: string): Option<string> =>
  id && id.length > 0 ? Option.some(id) : Option.none();

// --- Pure function: transformasi ---
const formatOrders = (orders: any[]) =>
  orders.map((o) => ({
    id: o.id,
    total: o.total,
    status: o.status,
    items: o.items ?? [],
  }));

// --- Repository (DIP) ---
interface OrderRepository {
  findByUserId(userId: string): Promise<any[]>;
}

class DBOrderRepository implements OrderRepository {
```

```
  async findByUserId(userId: string): Promise<any[]> {
    const result = await db.query(
      'SELECT * FROM orders WHERE user_id = $1 ORDER BY created_at DESC',
      [userId]
    );
    return result.rows ?? [];
  }
}

// --- Service (composition + strategy) ---
class OrderService {
  constructor(private repo: OrderRepository) {}

  async getUserOrders(userId: string): Promise<Result<any[], string>> {
    // Validasi pure
    const validId = validateUserId(userId);
    if (validId.isNone()) return Result.failure('Invalid user ID');

    // Ambil data dari repository
    const orders = await this.repo.findByUserId(validId.getOrElse(''));

    // Format pake pure function
    const formatted = pipe(formatOrders)(orders);

    return Result.success(formatted);
  }
}

// --- Mastra tool dengan Service ---
const orderService = new OrderService(new DBOrderRepository());

// const getUserOrdersTool = new MastraTool({
//   name: 'getUserOrders',
//   description: 'Ambil daftar order berdasarkan user ID',
//   execute: async ({ data }: { data: { userId: string } }) => {
//     const result = await orderService.getUserOrders(data.userId);
//
//     if (result.isFailure()) {
//       return { error: result.getOrElse('Unknown error') };
//     }
//
//     return { orders: result.getOrThrow() };
//   },
// });
```

Ringkasan

Konsep	Gampangnya	Kapan Pake
Pure Function	Input sama = output sama	Semua fungsi — bikin prediktabel
Immutability	Data jangan diubah, bikin baru	State management, Redux, data sharing
Composition	Gabung fungsi kecil jadi besar	Pipeline data, transformasi sequential
Currying	Set parameter dikit-dikit	Middleware factory, partial application
Option/Maybe	Handle null tanpa error	Data yang mungkin nggak ada
Result	Handle error tanpa throw	Operasi yang bisa gagal (IO, network)

Latihan

Soal 1 — Pure Function & Immutability

Refactor fungsi ini jadi pure dan immutable:

```
const cart = [];

function addItem(name: string, price: number) {
  cart.push({ name, price });
  return cart;
}

function applyDiscount(percent: number) {
  for (const item of cart) {
    item.price = item.price * (1 - percent / 100);
  }
  return cart;
}
```

Soal 2 — Composition

Buat pipeline format teks pake `pipe`:

1. `trim` — hapus spasi ujung
2. `capitalize` — kapitalisasi huruf pertama
3. `addPeriod` — tambah titik di akhir kalau belum ada

```
// Contoh: "  hello world " → "Hello world."
```

Soal 3 — Currying

Buat curried function `createApiClient(baseURL) → (endpoint) → (token) → Promise<Response>` yang pake `fetch`.

```
// const api = createApiClient('https://api.example.com');
// const users = await api('/users')('token123');
```

Soal 4 — Monad

Kamu punya `getUserByEmail(email)` dan `getOrdersByUserId(id)`. Keduanya bisa return null. Pake Option monad buat nge-chain mereka tanpa null check bersarang.

```
function getUserByEmail(email: string): Option<User> { /* ... */ }
function getOrdersByUserId(id: number): Option<Order[]> { /* ... */ }

// Chain: getUserByEmail → ambil id → getOrdersByUserId
const result = // ... implement
```

Soal 5 — Real World: Refactor Route

Refactor route Express berikut pake pure functions + Result monad + DIP:

```
app.post('/api/orders', async (req, res) => {
  try {
    const { userId, items } = req.body;

    if (!userId || !items || !items.length) {
      return res.status(400).json({ error: 'Invalid payload' });
    }

    const total = items.reduce((sum: number, item: any) => sum + item.price * it
    const order = await db.query(
      'INSERT INTO orders (user_id, total, status) VALUES ($1, $2, $3) RETURNING
      [userId, total, 'pending']
    );

    const result = await fetch('https://payment-api.com/charge', {
      method: 'POST',
      body: JSON.stringify({ orderId: order.rows[0].id, amount: total }),
    });

    if (!result.ok) throw new Error('Payment failed');

    await emailService.send(userId, 'Order confirmed');

    res.json({ success: true, data: order.rows[0] });
  } catch (err) {
    res.status(500).json({ error: 'Something went wrong' });
```

```
  }
});
```

Functional programming + design patterns = kode yang bersih, aman, dan gampang di-test. Latihan ini bakal sering kamu temui di kerjaan sehari-hari.